

Operating Systems and Concurrency

Processes 3: Threads

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen, Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham

United Kingdom

2026

Recap

Last Lecture

- **Types of schedulers:** preemptive/non-preemptive, long/medium/short term)
- Performance **evaluation criteria**
- Scheduling **algorithms:** FCFS, SJF, Round Robin, Priority Queues

CPU burst time

Common question from last time

	FCFS Position	CPU burst time	Priority
Process A	1	67	1 (high)
Process B	2	37	1 (high)
Process C	3	14	2 (low)
Process D	4	16	2 (low)

Definition (CPU Burst Time)

The **CPU burst time** is the amount of time the process needs to complete execution.

Goals for Today

Overview

- 1 Threads vs. processes
- 2 Different thread implementations
- 3 POSIX Threads (PThreads)

Threads

Threads from an OS Perspective

- A process consists of two **fundamental units**
 - **Resources**:
 - A logical address space containing the process image (program, data, heap, stack)
 - Files, I/O devices, I/O channels, ...
 - **Execution trace**, i.e., instructions that get run on the CPU

Threads

Threads from an OS Perspective

- A process consists of two **fundamental units**
 - **Resources:**
 - A logical address space containing the process image (program, data, heap, stack)
 - Files, I/O devices, I/O channels, ...
 - **Execution trace**, i.e., instructions that get run on the CPU
- A process can **share its resources** between **multiple execution traces**

Threads

Threads from an OS Perspective

- A process consists of two **fundamental units**
 - **Resources:**
 - A logical address space containing the process image (program, data, heap, stack)
 - Files, I/O devices, I/O channels, ...
 - **Execution trace**, i.e., instructions that get run on the CPU
- A process can **share its resources** between **multiple execution traces**
- Therefore it makes sense to introduce an **abstraction for an execution trace - threads.**

Threads

Threads from an OS Perspective

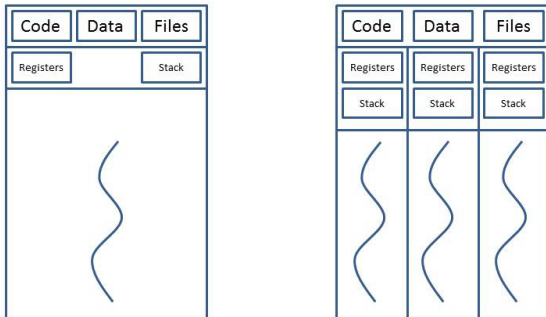


Figure: Single threaded process (left), multi-threaded process (right)

Threads

Threads from an OS Perspective

- Every thread has its own **execution context** (e.g. program counter, stack, registers)
- All threads have **access** to the process' **shared resources**
 - File handles: one thread opens a file, all threads of the same process can access the file
 - Global variables, heap memory, ...

Threads

Threads from an OS Perspective

- Every thread has its own **execution context** (e.g. program counter, stack, registers)
- All threads have **access** to the process' **shared resources**
 - File handles: one thread opens a file, all threads of the same process can access the file
 - Global variables, heap memory, ...
- Similar to processes, threads have:
 - **States and transitions** (new, running, blocked, ready, terminated)
 - A **thread control block (TCB)**
 - A **thread table** of TCBs
 - Each thread has a **thread id (TID)** (the term TID seems less common in practice than PID).

Threads

Threads from an OS Perspective

Processes	Threads
Address space	Program Counter
Global variables	Registers
Open files	Stack
Child processes	

Table: Process versus thread resources

Threads

Threads from an OS Perspective

- Threads ^{bring} incur **less overhead** to create/terminate/switch (address space remains the same for threads of the same process)
- Some CPUs have direct **hardware support** for **multi-threading**
 - Typically, they can offer up to 8 hardware threads per core

Threads

Threads from an OS Perspective

Share more resources

- **Inter-thread communication** is easier/faster than **interprocess** communication as threads share memory by default.
- **No protection boundaries** are required in the address space. Threads are assumed to be cooperating, with a common goal.
- Coordinating access to shared resources is **very complicated** - this is the topic of **synchronisation**.

Threads

Why Use Threads

- Multiple **related activities** apply to the **same resources**, these resources should be accessible/shared
- Processes will often contain multiple **blocking tasks**
 - I/O operations (thread blocks, **interrupt** marks completion)
 - Memory access: pages faults result in blocking
- Such activities should be carried out in **parallel/concurrently**
- **Application examples:** webserver, spreadsheets, word processors, anything with a user interface,...

Threads

OS Implementations of Threads

- **User** threads
- **Kernel** threads
- **Hybrid** implementations

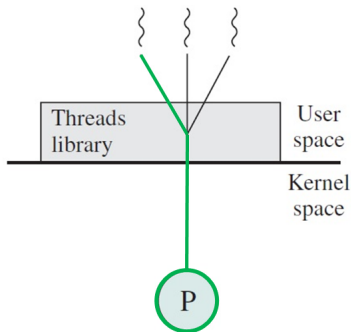
User Threads

Many-to-One

- **Thread management** (creating, destroying, scheduling, thread control block manipulation) is carried out in user space with the help of a **user library**
- The process maintains a **thread table** managed by the runtime system without the kernel's knowledge

User Threads

Many-to-One

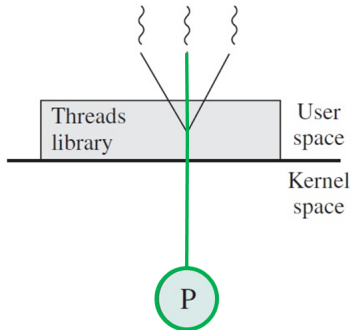


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

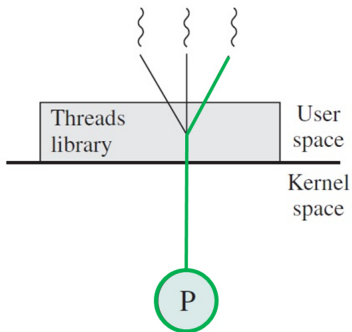


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

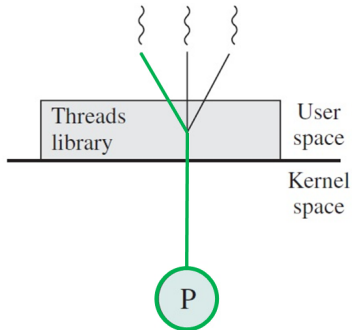


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

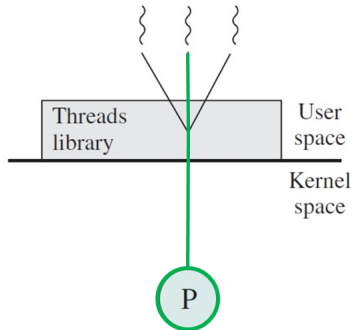


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

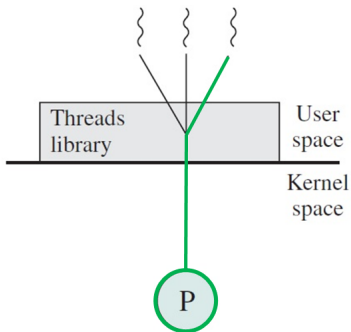


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

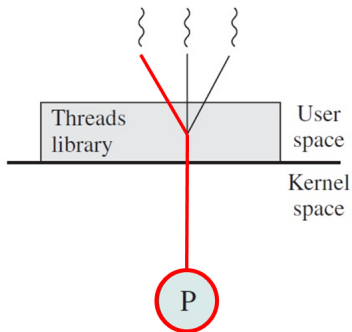


(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One



(a) Pure user-level

Figure: User threads (Stallings)

User Threads

Many-to-One

- Advantages:
 - Threads are in user space (i.e., no mode switches required)
 - Full control over the thread scheduler
 - OS independent (threads can run on OS that do not support them)
- Disadvantages:
 - Blocking system calls suspend the entire process (user threads are mapped onto a single process, managed by the kernel)
 - No true parallelism (a process is scheduled on a single CPU)
 - Non-preemptive as no mechanism such as interrupts

User Threads

Many-to-One

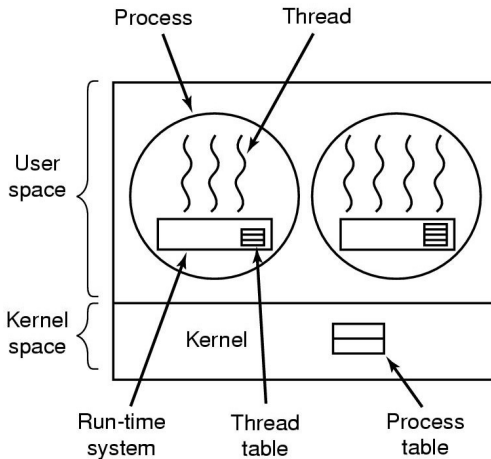


Figure: User threads (Tanenbaum 2014)

Kernel Threads

One-to-One

- The **kernel manages** the threads, user application accesses threading facilities through **API and system calls**
 - **Thread table** is in the kernel, containing thread control blocks (subset of process control blocks)
 - If a **thread blocks**, the kernel chooses a thread from same or different process
- Advantages:
 - **True parallelism** can be achieved
 - **Preemptive**
 - No run-time system needed in user space
- Frequent **mode switches** take place, resulting in lower performance
- Windows and Linux apply this approach

Kernel Threads

One-to-One

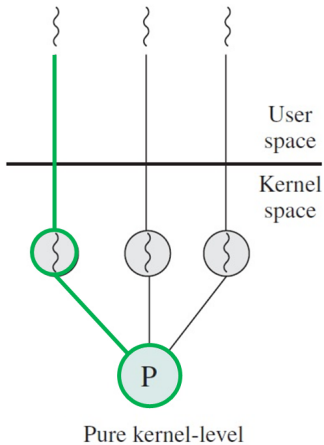


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

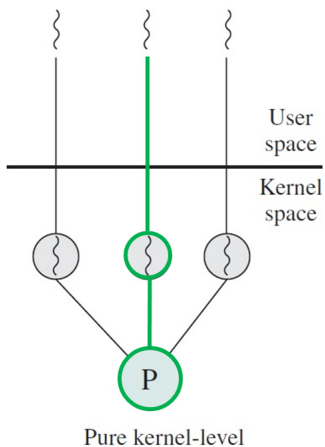


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

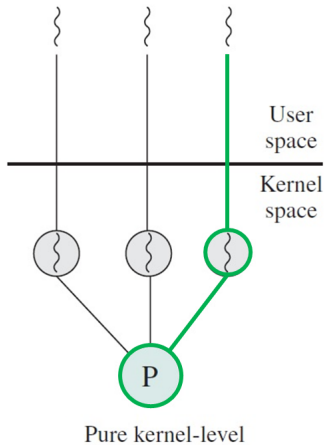


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

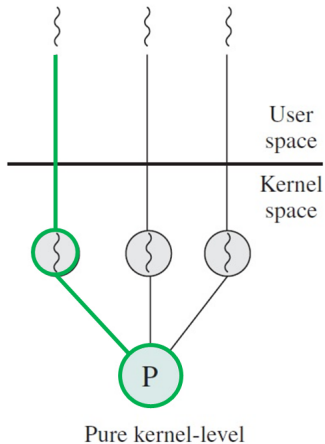


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

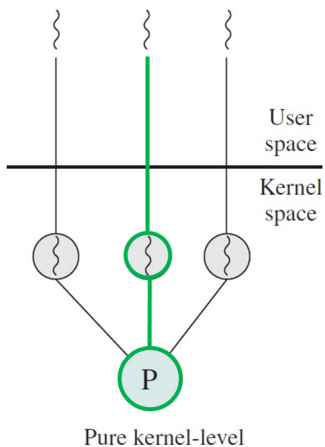


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

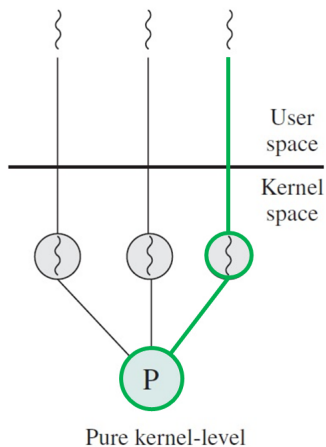


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

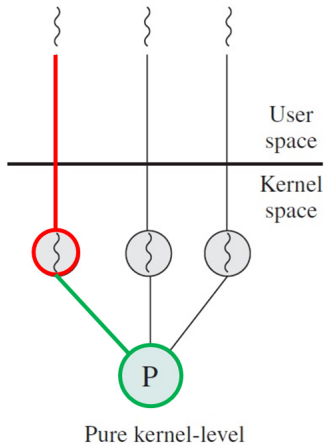


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

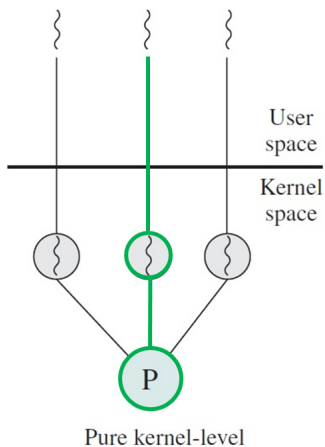


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

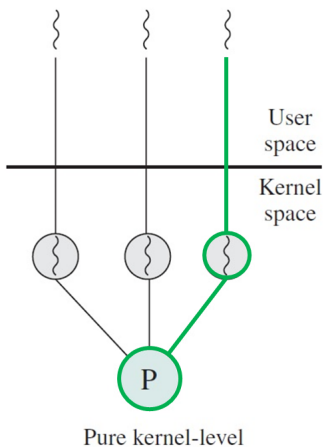


Figure: Kernel threads (Stallings 2014)

Kernel Threads

One-to-One

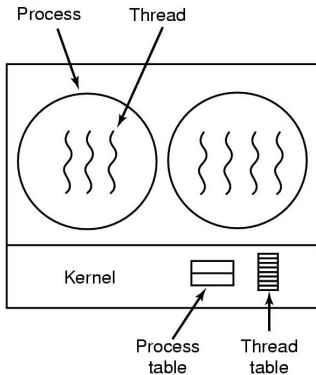


Figure: Kernel threads (Tanenbaum 2014)

Performance

User Threads vs. Kernel Threads vs. Processes

- Null fork: the overhead in creating, scheduling, running and terminating a null process/thread
- Signal wait: overhead in synchronising threads

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

Figure: Comparison, in μs (Stallings)

Hybrid Implementations

Many-to-Many

- User threads are **multiplexed** onto kernel threads
 - Kernel sees and schedules the kernel threads (a limited number)
 - User application sees user threads and creates/schedules these (an “unrestricted” number)
- 两层 scheduler

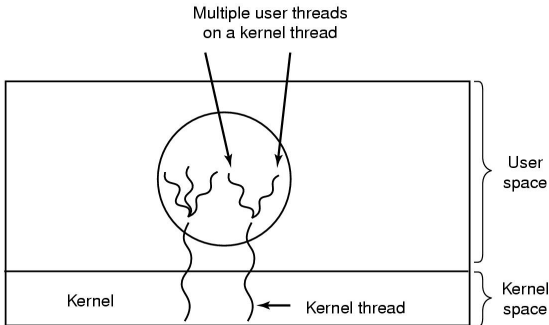


Figure: Kernel threads (Tanenbaum 2014)

Comparison

Thread Implementations

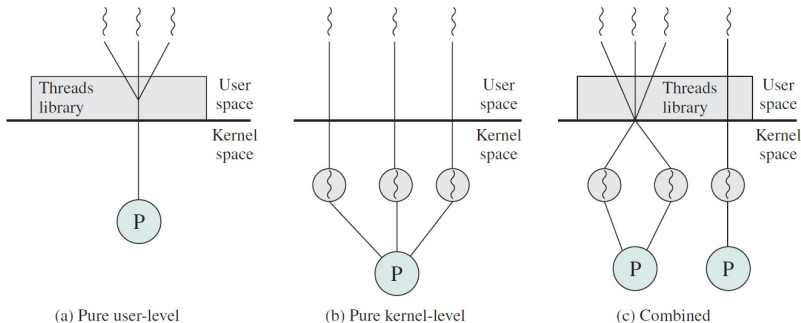


Figure: Comparison (Stallings)

User-level threads are preferred when low overhead and flexible user-space scheduling are more important than true parallelism.

Kernel-level threads are preferred when blocking operations, preemption, and true multicore parallelism are required.

Exam 2013-2014: In which situations would you favour user level threads? In which situation would you definitely favour kernel level threads?

Thread Management

Libraries

- Thread libraries provide an **API/interface for managing threads** (e.g. creating, running, destroying, synchronising, etc.)
- Thread libraries can be implemented:
 - Entirely in **user space** (i.e. user threads)
 - Based on **system calls**, i.e., rely on the kernel for thread implementations
- Examples of thread APIs include **POSIX's PThreads, Windows Threads, and Java Threads**
 - The PThread specification can be implemented as user or kernel threads

POSIX Threads

Overview

- POSIX threads are a **specification** that “**anyone**” **can implement**, i.e., it defines a set of APIs (function calls, over 60 of them) and what they do
- Core thread management functions we shall use include:

Function Call	Summary
<code>pthread_create</code>	Create new thread
<code>pthread_exit</code>	Exit existing thread
<code>pthread_join</code>	Wait for thread with ID

Table: PThread examples

- More detailed descriptions can be found using `man function_name` on the command line

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

POSIX Threads

Example

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for(int i = 0; i < THREADS; i++) {
        args[i] = i;
        if(pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for(int i = 0; i < THREADS; i++) 主线程等待 threads[i] 执行结束
        pthread_join(threads[i], NULL);
}
```

Test your understanding

1. Each thread needs its own function calls, local variables and execution state.
 - If the threads in a process share the same memory, why do they have independent stacks?
 - Is it always necessary to call `pthread_exit` when ending a thread?
 - What is the minimum number of threads a process can have?
 - Can user threads make good use of concurrent hardware?
2. No, returning from the thread function also terminates it.
 3. One thread. Every running process must have at least one thread of execution, usually called the main thread.
 4. In a pure many-to-one user-thread model, no. Multiple user threads are mapped onto a single kernel thread, so the kernel schedules only one execution entity. Therefore they cannot run truly in parallel on multiple CPU cores.
However, user threads can still provide concurrency through user-space scheduling, and in a hybrid many-to-many implementation they can make use of multiple kernel threads.

Processes 3 – Core Revision Summary

Threads vs processes · shared memory · blocking · synchronisation · user/kernel/hybrid implementations · POSIX PThreads
 ♦ source-marked / annotated content · discussion clarifications integrated

1. Topic Overview

This lecture separates a process into **resources** and an **execution trace**. A **thread** is the abstraction for an execution trace, allowing one process to contain multiple independently scheduled activities while sharing process resources. The lecture then compares **user-level, kernel-level, and hybrid thread implementations** before introducing POSIX Threads (PThreads).

Lecture logic: process resources + execution trace → thread abstraction → shared/private thread state → why threads → implementation models → PThreads API

2. Core Concepts

Process resources vs. thread execution state

- ▶ ♦ A process contains **resources** (logical address space, files, I/O devices/channels) plus an **execution trace**. Multiple execution traces can share the same process resources.
- ▶ ♦ Each thread has its own **execution context: program counter, registers, and stack**.
- ▶ ♦ Threads in the same process share **address space, global variables, heap memory, and open files**. A file opened by one thread can therefore be accessed by the others.
- ▶ ♦ Like processes, threads have states (new, ready, running, blocked, terminated), a **Thread Control Block (TCB)**, a thread table, and a thread ID (TID).

Process-level resources (shared)	Thread-level state (private per thread)
Address space · global variables · heap	Program Counter (PC)
Open files · I/O resources	Registers
Child-process relationship / process resources	Stack (function calls, return addresses, locals)

Why threads?

- ▶ ♦ Threads have lower create/terminate/switch overhead because threads of the same process keep the **same address space**.
- ▶ ♦ **Inter-thread communication is easier/faster** than interprocess communication because threads **share memory by default**.
- ▶ ♦ **No protection boundary** is normally required between cooperating threads inside one address space. This makes sharing convenient, but also means one thread can affect data used by another.
- ▶ ♦ Coordinating simultaneous access to shared resources is difficult; this is the topic of **synchronisation**.

Blocking tasks and responsiveness

- ▶ ♦ A process may contain multiple **blocking tasks**: I/O can block a thread until an interrupt reports completion; a page fault can also block while memory is made available.
- ▶ ♦ Using multiple threads lets other activities continue while one is blocked, so related activities can be carried out **concurrently / in parallel**. Examples: web servers, spreadsheets, word processors, and GUI applications.

Key distinction: concurrency = multiple activities make progress over the same interval; **parallelism** = activities actually execute simultaneously on different hardware execution units.

3. Thread Implementation Models

A) User-level threads — Many-to-One

- ▶ ♦ Thread creation, destruction, scheduling, and TCB manipulation happen in **user space** via a user library; the runtime maintains a thread table without the kernel knowing about individual user threads.
- ▶ ♦ **Advantages:** no thread-management mode switches, full control over the user-space scheduler, and OS independence.
- ▶ ♦ **Disadvantages:** a blocking system call can suspend the whole process; there is **no true parallelism** in pure many-to-one; scheduling is non-preemptive because the user library lacks mechanisms such as kernel interrupts.

B) Kernel-level threads — One-to-One

- ▶ ♦ The **kernel manages the threads**; applications access threading through APIs and system calls. The kernel holds the thread table / TCBs.
- ▶ ♦ If one thread blocks, the kernel can schedule another thread from the same or a different process.
- ▶ ♦ **Advantages:** true multicore parallelism, preemptive scheduling, and no user-space runtime system required. **Trade-off:** frequent mode switches increase overhead. Windows and Linux use this approach.

C) Hybrid — Many-to-Many

- ▶ ♦ **User threads are multiplexed onto kernel threads.** The application/runtime can create and schedule a large number of user threads, while the kernel sees and schedules only a limited set of kernel threads.

Two scheduling layers: user runtime schedules user threads → kernel threads; kernel scheduler schedules kernel threads → CPU cores.

Implementation comparison

Model	Who schedules?	Main strength	Main limitation
Many-to-One	User runtime	Low overhead; flexible user scheduler	Blocking can stall process; no true multicore parallelism
One-to-One	Kernel	Preemption + true parallelism; block one ≠ block all	Higher mode-switch / kernel-management overhead
Many-to-Many	Runtime + kernel	Many lightweight user threads with multiple kernel execution paths	More complex two-level management

Measured overhead example from the lecture (µs)

Operation	User-level threads	Kernel-level threads	Processes
Null fork	34	948	11,300
Signal wait	37	441	1,840

Null fork measures create/schedule/run/terminate overhead; **signal wait** measures synchronisation overhead. The table illustrates why user-level management can be much cheaper.

Exam 2013–2014: choosing the model

- ▶ ♦ **Favour user-level threads** when low overhead and flexible user-space scheduling matter more than true parallelism, and blocking operations are limited.
- ▶ ♦ **Definitely favour kernel-level threads** when blocking operations, preemption, or true multicore parallelism are required.

4. Thread Management & POSIX Threads (PThreads)

Thread libraries and PThreads

- ▶ ♦ Thread libraries provide APIs for creating, running, destroying, and synchronising threads. They may be implemented entirely in user space or via kernel system calls.
- ▶ ♦ Examples include POSIX PThreads, Windows Threads, and Java Threads. **PThreads is a specification** of API behaviour; it can be implemented using user or kernel threads.

PThread call	Purpose
pthread_create	Create a new thread
pthread_exit	Exit the current thread explicitly
pthread_join	Wait for a thread identified by pthread_t

Core example — what each line is doing

```
#include <pthread.h>
#include <stdio.h>
#define THREADS 10

void* hello(void* arg) {
    printf("Hello from thread %d\n", *((int*)arg));
    return 0;
}

int main() {
    int args[THREADS] = { 0 };
    pthread_t threads[THREADS];
    for (int i = 0; i < THREADS; i++) {
        args[i] = i;
        if (pthread_create(threads + i, NULL, hello, args + i)) {
            printf("Creating thread %d failed\n", i); return -1;
        }
    }
    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);
}
```

Code anatomy

- ▶ **int args[THREADS] = {0};** creates 10 independent integer slots. The loop writes 0...9 so each thread receives a stable thread number.
- ▶ **pthread_t threads[THREADS];** stores the identifiers returned for the 10 created threads.
- ▶ **pthread_create(threads+i, NULL, hello, args+i)** means: store the new ID in threads[i]; use default attributes; start in hello; pass &args[i] as the argument.
- ▶ ***((int*)arg)** casts the generic void* back to int* and dereferences it to obtain the thread number.
- ▶ **pthread_join(threads[i], NULL)** makes the main thread wait until that worker thread finishes.

Discussion clarification: passing a separate **&args[i]** to each thread avoids the classic mistake of giving every thread the same changing loop variable &i.

How does a thread end?

- ▶ ♦ Calling **pthread_exit** is **not always necessary**. Returning from the thread start routine also terminates that thread.
- ▶ ♦ A running process has at least **one thread** — normally the main thread — even if no additional PThreads are created.

5. Comparison & Common Confusions

Concept A	Concept B	Key Difference / Revision Point
Process	Thread	Process is the resource container; a thread is an execution trace / execution unit inside it.
Shared process memory	Independent thread stack	Threads share the address space, but each needs private calls, locals, return addresses, PC and registers.
Concurrency	Parallelism	Concurrency = overlapping progress; parallelism = simultaneous execution on separate hardware execution units.
Blocking task	Whole-process blocking	With kernel threads, another thread can run when one blocks; pure many-to-one user threads can cause the whole process to stall on a blocking syscall.
User-level threads	Kernel-level threads	User: cheaper and user-scheduled, but no true parallelism in many-to-one. Kernel: preemptive + multicore parallelism, but higher overhead.
Many-to-One	Many-to-Many	Many-to-One maps all user threads to one kernel execution entity; Many-to-Many multiplexes them over multiple kernel threads.
pthread_create	pthread_join	create starts a thread; join waits for an existing thread to terminate.

6. Test Your Understanding — Answers

1) If threads share the same memory, why do they have independent stacks?

- ▶ ♦ Because each thread executes independently. Its stack stores its own function calls, return addresses, parameters, and local variables; sharing one stack would corrupt the threads' execution states.

2) Is pthread_exit always necessary?

- ▶ ♦ No. Returning from the thread function also terminates the thread. pthread_exit is useful when termination is made explicit.

3) Minimum number of threads in a process?

- ▶ ♦ **One.** Every running process needs at least one thread of execution, normally the main thread.

4) Can user threads make good use of concurrent hardware?

- ▶ ♦ In a pure **many-to-one** model, no: the kernel sees one execution entity, so multiple user threads cannot truly run on multiple CPU cores. They can still provide concurrency through user-space scheduling; a hybrid many-to-many design can exploit multiple kernel threads.

7. Exam / Revision Focus

Must Know

Definitions: thread, execution context, TCB/TID, blocking, synchronisation, user/kernel/hybrid threads, Many-to-One / One-to-One / Many-to-Many.

Shared vs private state: shared address space/global/heap/files; private PC/registers/stack.

PThreads: **pthread_create**, **pthread_exit**, **pthread_join**.

Must Understand

Why shared memory makes inter-thread communication cheap but creates synchronisation problems.

Why blocking operations motivate multiple threads, and why kernel threads handle blocking more robustly than pure many-to-one user threads.

Why user-level threads are cheaper, but kernel-level threads are required for preemption and true multicore parallelism.

How the four arguments to pthread_create connect a thread ID, attributes, start routine, and argument.

Easy to Confuse

"Threads share memory" does **not** mean they share stacks/registers/program counters.

User-level threads can be **concurrent** without being truly **parallel** in the many-to-one model.

PThreads is an **API specification**; it does not by itself require a user-level or kernel-level implementation.

pthread_join waits; it does not create or terminate the target thread.

Source emphasis incorporated: marked/underlined statements and annotations on pp.5–14, 16, 24, 26, 37–41, 43–53; discussion clarifications on shared memory/protection, blocking tasks, hybrid scheduling, implementation choice, PThreads code, and self-test answers.